OS Lab

Practical-12

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define ITEMS 100000

int buffer_size;
int *buffer;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
   for (int i = 0; i < ITEMS; i++)
   {
      // Wait for an empty slot
      sem_wait(&empty);

      // Enter critical section
      pthread_mutex_lock(&mutex);

      buffer[in] = i;
      in = (in + 1) % buffer_size;

      pthread_mutex_unlock(&mutex);
```

```
        // Increase number of full slots
        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    long sum = 0;

    for (int i = 0; i < ITEMS; i++)
    {
        // Wait for an available item
        sem_wait(&full);

        // Enter critical section
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        out = (out + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        // Increase number of empty slots
        sem_post(&empty);

        sum += item;
    }

    printf("Consumer sum: %ld\n", sum);

    return NULL;
}

int main(int argc, char *argv[])
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    if (argc != 2)
    {
        printf("Usage: %s <buffer_size>\n", argv[0]);
```

```
    return 1;
}

buffer_size = atoi(argv[1]);

if (buffer_size <= 0)
{
    printf("Buffer size must be greater than 0.\n");
    return 1;
}

buffer = malloc(buffer_size * sizeof(int));

if (buffer == NULL)
{
    perror("Memory allocation failed");
    return 1;
}

// Initialize semaphores
sem_init(&empty, 0, buffer_size);
sem_init(&full, 0, 0);

// Initialize mutex
pthread_mutex_init(&mutex, NULL);

pthread_create(&producer_thread, NULL, producer, NULL);
pthread_create(&consumer_thread, NULL, consumer, NULL);

pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);

printf("Producer-Consumer completed successfully.\n");
printf("Buffer size: %d\n", buffer_size);
printf("Items produced: %d\n", ITEMS);
printf("Items consumed: %d\n", ITEMS);

sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

free(buffer);

return 0;
```

}

```
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ nano pthread_race.c
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ gcc pthread_race.c -o pthread_race -pthread
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./pthread_race
Initial counter: 0
Expected counter: 500000
Actual counter: 175412
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./pthread_race
Initial counter: 0
Expected counter: 500000
Actual counter: 147510
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./pthread_race
Initial counter: 0
Expected counter: 500000
Actual counter: 163427
```

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
   printf("Thread 1: Locking Resource 1\n");
   pthread_mutex_lock(&resource1);

   printf("Thread 1: Resource 1 locked\n");

   sleep(1);

   printf("Thread 1: Waiting for Resource 2\n");
   pthread_mutex_lock(&resource2);

   printf("Thread 1: Resource 2 locked\n");

   pthread_mutex_unlock(&resource2);
   pthread_mutex_unlock(&resource1);

   return NULL;
}

void *thread2(void *arg)
{
   printf("Thread 2: Locking Resource 2\n");
   pthread_mutex_lock(&resource2);
```

```
    printf("Thread 2: Resource 2 locked\n");

    sleep(1);

    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Resource 1 locked\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}
```

```
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ nano pthread_mutex.c
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ gcc pthread_mutex.c -o pthread_mutex -pthread
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./pthread_mutex
Initial counter: 0
Expected counter: 500000
Actual counter: 500000
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./pthread_mutex
Initial counter: 0
Expected counter: 500000
Actual counter: 500000
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./pthread_mutex
Initial counter: 0
Expected counter: 500000
Actual counter: 500000
```

```
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./producer_consumer_sem 1
Consumer sum: 4999950000
Producer-Consumer completed successfully.
Buffer size: 1
Items produced: 100000
Items consumed: 100000
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./producer_consumer_sem 5
Consumer sum: 4999950000
Producer-Consumer completed successfully.
Buffer size: 5
Items produced: 100000
Items consumed: 100000
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./producer_consumer_sem 10
Consumer sum: 4999950000
Producer-Consumer completed successfully.
Buffer size: 10
Items produced: 100000
Items consumed: 100000
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./producer_consumer_sem 50
Consumer sum: 4999950000
Producer-Consumer completed successfully.
Buffer size: 50
Items produced: 100000
Items consumed: 100000
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./producer_consumer_sem 100
Consumer sum: 4999950000
Producer-Consumer completed successfully.
Buffer size: 100
Items produced: 100000
Items consumed: 100000
```

```
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ time ./producer_consumer_sem 1
Consumer sum: 4999950000
Producer-Consumer completed successfully.
Buffer size: 1
Items produced: 100000
Items consumed: 100000

real    0m3.553s
user    0m0.114s
sys     0m2.723s
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ time ./producer_consumer_sem 50
Consumer sum: 4999950000
Producer-Consumer completed successfully.
Buffer size: 50
Items produced: 100000
Items consumed: 100000

real    0m0.049s
user    0m0.030s
sys     0m0.039s
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ time ./producer_consumer_sem 100
Consumer sum: 4999950000
Producer-Consumer completed successfully.
Buffer size: 100
Items produced: 100000
Items consumed: 100000

real    0m0.048s
user    0m0.032s
sys     0m0.036s
```

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
   printf("Thread 1: Locking Resource 1\n");
   pthread_mutex_lock(&resource1);
```

```
    printf("Thread 1: Resource 1 locked\n");

    sleep(1);

    printf("Thread 1: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Resource 2 locked\n");

    printf("Thread 1: Using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 1: Resources released\n");

    return NULL;
}

void *thread2(void *arg)
{
    // IMPORTANT:
    // Thread 2 also locks Resource 1 first.

    printf("Thread 2: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Resource 1 locked\n");

    sleep(1);

    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Resource 2 locked\n");

    printf("Thread 2: Using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 2: Resources released\n");

    return NULL;
```

```
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Program completed without deadlock.\n");

    return 0;
}
```

```
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ nano deadlock.c
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ nano deadlock.c
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ gcc deadlock.c -o deadlock -pthread
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./deadlock
Thread 1: Locking Resource 1
Thread 1: Resource 1 locked
Thread 2: Locking Resource 2
Thread 2: Resource 2 locked
Thread 1: Waiting for Resource 2
Thread 2: Waiting for Resource 1
^C
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ nano deadlock_prevention.c
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ gcc deadlock_prevention.c -o deadlock_prevention -pthread
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$ ./deadlock_prevention
Thread 1: Locking Resource 1
Thread 1: Resource 1 locked
Thread 2: Locking Resource 1
Thread 1: Locking Resource 2
Thread 1: Resource 2 locked
Thread 1: Using both resources
Thread 1: Resources released
Thread 2: Resource 1 locked
Thread 2: Locking Resource 2
Thread 2: Resource 2 locked
Thread 2: Using both resources
Thread 2: Resources released
Program completed without deadlock.
lahari@DESKTOP-AIC33F7:~/OSLab/file_io$
```